Enterprise Knowledge Assistant

System Design Document

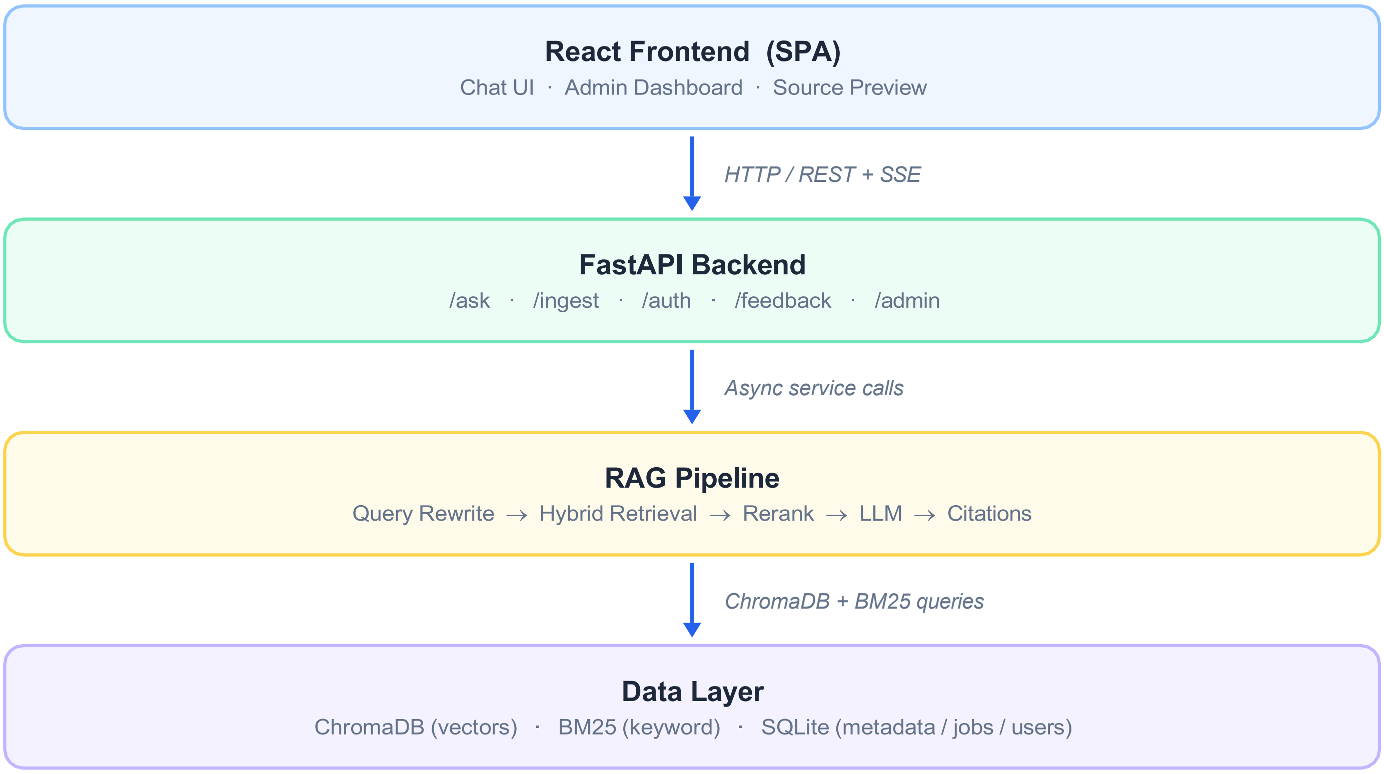
June 2026

Python 3.11 FastAPI ChromaDB Ollama

1. System Overview

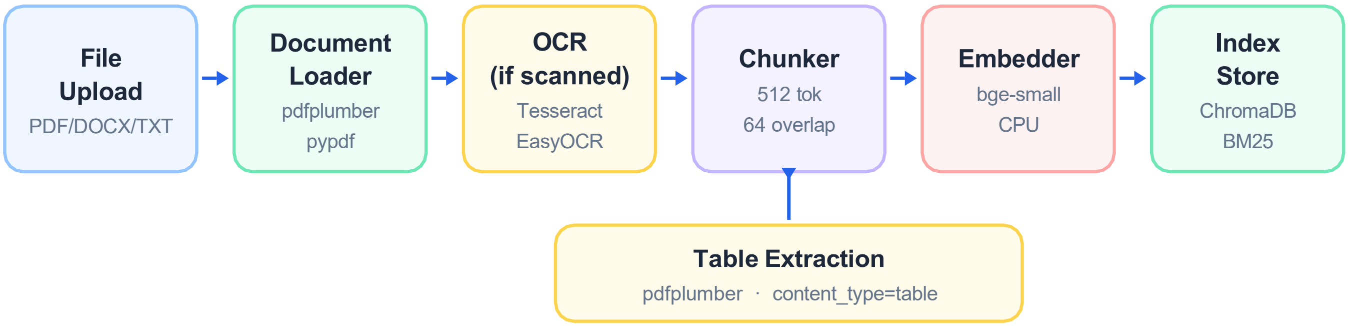
The Enterprise Knowledge Assistant (EKA) is a fully local RAG system that lets employees ask natural-language questions over a company's internal document library and receive accurate, cited answers. Every component runs on-premise — no external API calls required.

2. High-Level Architecture



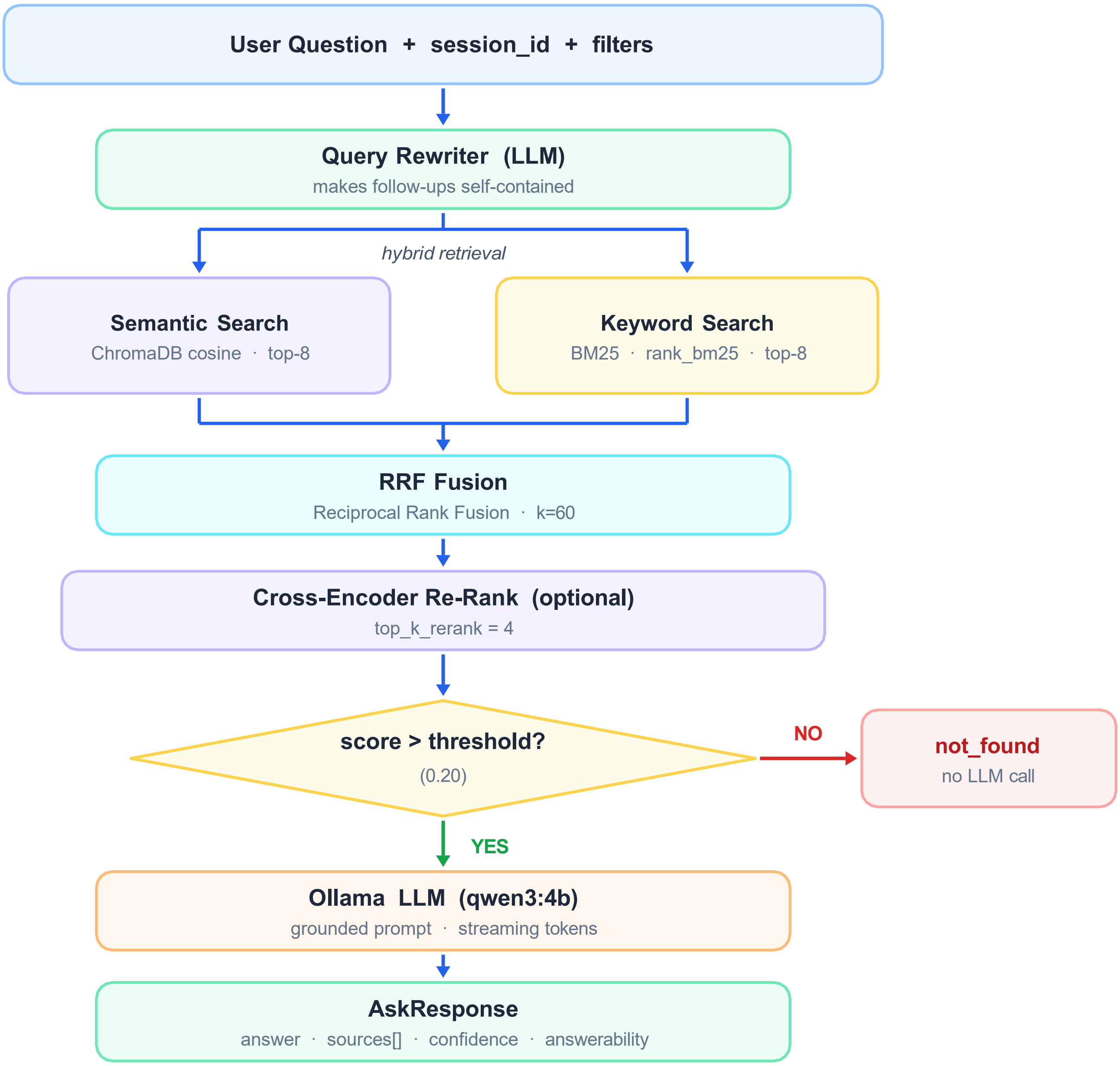
- All traffic stays on-premise. Ollama runs the LLM locally — no data leaves the server.
- The /ask/stream endpoint returns ndjson SSE events for real-time token rendering.
- JWT Bearer tokens (HS256) gate every endpoint. Role-based access: admin / employee.

3. Document Ingestion Pipeline



Each chunk stores: filename · page_number · department · document_type · tags · policy_version · access_roles · content_type

4. Query & Answer Flow



Section 5

Component Map

The EKA back-end is organized into 12 focused modules, each owning a single responsibility. Every module is independently testable and can be upgraded without touching the rest of the pipeline.

Module / Path	Primary Role	Key Details
`app/api/main.py`	**FastAPI Entry Point**	Registers routers, configures CORS, runs startup hooks: BM25 rebuild and Chroma warm-up.
`app/auth/`	**Auth & RBAC**	JWT issuance/verification (HS256), bcrypt hashing, role-based access: admin / employee.
`app/api/routes_ask.py`	**Q&A; Endpoints**	POST /ask (blocking) and POST /ask/stream (SSE). Orchestrates the full RAG pipeline.
`app/api/routes_ingest.py`	**Document Ingestion**	File upload, directory scan, chunk deletion, and async job tracker for long ingestion.
`app/api/routes_sources.py`	**Source Preview**	Returns chunk text for UI citations. Enforces access_roles per user role.
`app/api/routes_feedback.py`	**User Feedback**	Thumbs-up/down endpoint. Persists to SQLite; deduplicates on msg_id.
`app/generation/`	**Answer Orchestrator**	Query rewrite → hybrid retrieval → rerank → LLM generation → citation assembly.
`app/indexing/embeddings.py`	**Embedding Model**	Wraps BAAI/bge-small-en-v1.5 via Sentence Transformers. CPU; swappable for GPU.
`app/indexing/chroma_store.py`	**Vector Store**	ChromaDB wrapper: filtered cosine similarity + metadata predicate push-down.
`app/indexing/bm25_store.py`	**Keyword Index**	rank_bm25 in-memory index rebuilt from Chroma metadata on startup.
`app/generation/ollama_client.py`	**LLM Client**	Async HTTP to Ollama, streaming tokens, fallback to smaller model on OOM.
`app/observability/`	**Tracing & Metrics**	Per-request structured logs with latency breakdowns per pipeline stage.

Section 6

Technology Choices

Every tool was selected to maximise on-premise operability and provide a clear upgrade path when scale demands it.

Layer	Choice	Why
Language	**Python 3.11**	Richest ML/NLP ecosystem; async-native; broad community support.
Web framework	**FastAPI + uvicorn**	Async-native, auto OpenAPI docs, Pydantic validation, minimal boilerplate.
LLM runtime	**Ollama qwen3:4b / 1.7b**	Fully local, zero API cost, streaming support, easy model switching.
Embeddings	**BAAI/bge-small-en-v1.5**	High retrieval quality; CPU-friendly; swappable to bge-base or bge-m3.
Vector store	**ChromaDB (embedded)**	Zero-config local persistence; server mode available for horizontal scaling.
Keyword index	**rank_bm25**	Lightweight pure-Python BM25; complements semantic for exact keyword matches.
Fusion	**Reciprocal Rank Fusion**	Combines ranked lists without score normalisation; proven recall gains.
Re-ranker	**Cross-encoder (optional)**	Significant precision gain on top-k; skipped by default to reduce CPU latency.
PDF parsing	**pdfplumber + pypdf**	Layout-aware extraction with table support; pypdf handles edge cases.
OCR	**Tesseract / EasyOCR**	Open-source; auto-triggered on low-text-density pages.
Auth	**JWT + bcrypt (jose)**	Stateless tokens; industry-standard; no external auth service required.
Persistence	**SQLite**	Zero-config; handles metadata, users, feedback, and job tracking.

Section 7

Scalability Considerations

The current design targets a single-server deployment — ideal for 50–500 users. Each bottleneck maps to a concrete upgrade path that does not require rewriting the application layer.

Bottleneck	Current Behaviour	Scale-out Path
Embedding throughput	CPU inference with bge-small (~50 docs/min).	GPU node (CUDA) + upgrade to bge-base or bge-m3.
LLM latency	Single Ollama process; one request at a time.	Multi-GPU Ollama cluster or cloud API fallback.
Vector search	ChromaDB embedded on local disk; single-process.	Chroma server mode · Pinecone · Weaviate · pgvector.
Keyword index	In-memory BM25 rebuilt from Chroma on startup.	Persist BM25 to disk; Elasticsearch for large corpora.
Session memory	In-process dict; lost on restart; not shared.	Redis or SQLite-backed store across Gunicorn workers.
API concurrency	Single uvicorn process; vertical limit.	Gunicorn multi-worker + asyncio; stateless workers.
Document storage	Local filesystem; no redundancy.	S3 / Azure Blob / GCS with pre-signed URLs.
Auth / access	SQLite + JWT; access_roles in Chroma metadata.	LDAP / SAML SSO; per-doc ACL in Chroma metadata.

Section 8

Horizontal Scale-Out Blueprint

When a single server is insufficient, the architecture moves to a stateless API tier behind a load balancer, with shared state externalised to Redis, ChromaDB Server, and an Ollama cluster.

Load Balancer (nginx / AWS ALB)
Routes all incoming API traffic evenly

API Worker 1 — FastAPI stateless	**API Worker 2 — FastAPI stateless**	**API Worker N — FastAPI stateless**
asyncio event loop, scales horizontally	asyncio event loop, scales horizontally	Add more workers as load increases

Redis	**ChromaDB Server**	**Ollama Cluster**
Session memory Conversation history	Vector store Shared collection	LLM inference Multi-GPU nodes

Section 9

Evaluation & Known Limitations

System health is tracked through four core metrics. Four known limitations are documented with recommended mitigations.

Evaluation Metrics

■ Answerability ratio

Fraction answered vs. not_found. Low ratio signals retrieval gaps. Target: >85%.

■ End-to-end latency

Wall-clock time to final token. Per-stage: rewrite+retrieval+generation. Target: <8 s.

■ User feedback score

Thumbs-up/down ratio via /feedback. Displayed on admin dashboard.

■ Retrieval precision

Cross-encoder score distribution for top-4 chunks. Low scores signal tuning needed.

Known Limitations

■ Ollama must run locally

No cloud LLM fallback by default. Proxy through OpenAI-compatible gateway for resilience.

■ BM25 rebuilt on startup

Startup time grows with corpus. Slow for >200k chunks. Mitigation: persist index to disk.

■ Session memory in-process

History lost on restart; not shared across workers without Redis.

■ JWT secret in .env

Must be rotated and moved to a secrets manager before production deployment.